Kick Start 2017 - Round D

Analysis: Sherlock and Matrix Game

Sherlock and The Matrix Game : Analysis

Brute force approaches

In the Small dataset, **N** (which is the size of arrays **A** and **B**) doesn't exceed 200. A naive bruteforce strategy is to actually create the matrix and then iterate over all possible submatrices and calculate the sum for each submatrix. Since the number of submatrices is $O(N^4)$ (observe that for every pair of cells, there exists a unique submatrix), naively iterating over each submatrix in O(*submatrix size*) is bound to run for years(it's a whopping $1.2 * 10^{15}$ operations for 20 test cases in the worst case). However, if we can calculate sum of each submatrix in constant time, the number of operations will reduce to $3 * 10^{10}$, which is feasible for a modern machine in minutes.

Inclusion-exclusion to the rescue

Quickly calculating submatrix sums of a matrix M can be done by maintaining another matrix P such that P(i, j) = sum of all cells M(a, b), such that $1 \le a \le i$ and $1 \le b \le j$. Matrix P can be built in O(*size of* M) time by using the recurrence P(i, j) = M(i, j) + P(i-1, j) + P(i, j-1) - P(i-1, j-1). Note that the first three terms on the right hand side count some of the cells twice, which is why we subtract off the fourth term. Further, the sum of a submatrix defined by top-left and bottom-right cells (a, b) and (c, d) can be calculated as P(c, d) - P(a-1, d) - P(c, b-1) + P(a-1, b-1). Again, we add the last term on right hand side to accomodate the double subtraction of some cells.

A binary search large approach

For the Large dataset, **N** could be up to 10^5, which implies that creating the matrix in-memory is not possible, let alone iterating over all possible submatrices. We can, however, use the fact that $M(i, j) = \mathbf{A}_i * \mathbf{B}_j$ to our advantage by expressing sum of cells in the submatrix defined by top-left and bottom-right cells (a, b) and (c, d), respectively, as $(\mathbf{A}_a + \mathbf{A}_{a+1} + ... + \mathbf{A}_c) * (\mathbf{B}_b + \mathbf{B}_{b+1} + ... + \mathbf{B}_d)$, i.e., product of sum of contiguous subarrays of **A** and **B**.

If we create two sorted lists U and V consisting of all possible subarray sums of **A** and **B**, respectively, we're trying to find the $\mathbf{K}^{\mathbf{th}}$ largest value among $U_i * V_j$ for all i, j. At this moment, we can exploit the fact that **K** doesn't exceed 10^5 by observing that we don't need to consider all possible values in U and V. We can work with the **K** largest and smallest values from both arrays(why the smallest? don't forget we also have negative values?) and be guaranteed that answer will be present in product of these values. At this point, we face two subproblems to finally solve this problem.

Subproblem 1

Given an array **A** of size **N**, find the **K** largest subarray sums of **A**. Note that the value of **K** is approximately O(**N**). If we can do this, finding the **K** smallest subarray sums is easy by reversing the signs of values in A and running the same algorithm.

Solution

We use binary search! The idea is to first find the $\mathbf{K^{th}}$ largest subarray sum by binary searching for it, and then build the actual subarray sums. To apply binary search for the $\mathbf{K^{th}}$ largest subarray sum, we need a function which can quickly count: how many subarray sums are less than X, for a given X?
Let us define P_i as sum of the first i elements of array **A**. Now, a simple idea is to iterate over i and fix the subarray start at position i. Now, we're trying to count possible j(≥ i) such that $P_j \geq P_i + X$. Note that the right size of the inequality is a constant for a fixed i. Things would've been easier if prefix sum array P had been an increasing sequence. However, that is not the case, since there are also negative values in array **A**. Suppose that we iterate over i from **N** to 1. At each step, we'll try to calculate the answer for i(which depends on all values $\mathbf{A}_j$ such that j ≥ i). Imagine we have a data structure DS which supports two operations:

1. Insert y : inserts y in the data structure.
2. Query y : returns the count of values present in data structure that are less than or equal to y.

Using this DS, our job can be done(we can do DS.Insert($\mathbf{A}_i$), add DS.Query($P_i + X$) to our answer, and then decrement i and so on). Such a data structure can be implemented efficiently using any balanced binary search trees such as splay trees. Such trees can handle both operations in O(log(*size of DS*)). If the same data structure can support iterating over all values present in it in increasing order, we can get all subarray sums less than the $\mathbf{K^{th}}$ smallest subarray sum in a similar way. Note that we don't need the "Query" operation for doing this. So, the complexity of solving this subproblem turns out to be O(log(*range of answer*) * **N** * log(**N**) + **K**).

Subproblem 2

Given two arrays **A** and **B** of size O(**N**), find the $\mathbf{K^{th}}$ largest among all possible values $\mathbf{A}_i * \mathbf{B}_j$, for all i, j.

Solution

Again, we can use binary search on our answer if we, given X, can count how many pairs i, j exist such that $\mathbf{A}_i * \mathbf{B}_j \geq X$.
Just as in to subproblem 1, the idea is to iterate over i, and then count all possible j such that $\mathbf{B}_j \geq X / \mathbf{A}_i$, which is easily doable using binary search if array **B** is kept in a sorted fashion.
However, note that it is a little trickier because of negative values. The complexity of solving this subproblem turns out to be (log(*range of answer*) * **N** * log(**N**)).

Kick Start 2017 - Round D

Analysis: Sightseeing

Go Sightseeing: Analysis

Small dataset

There are at most 15 cities in which you can go sightseeing, and in each of those cities, we can either go sightseeing or not, so there are only 2^{15} possibilities to consider. A brute force approach that tries each of those 2^{15} possibilities is fast enough. For each possibility, we can compute the earliest possible time of arrival in city **N** and then find the possibility with the greatest number of sightseeing trips that arrives at city **N** on time.

It can be observed that the time of arrival in any city i + 1 is only dependent on T_w, the time at which we start waiting for the bus in city i. This can be expressed as *Arrival*(i + 1, T_w) = T' + $\mathbf{D_i}$ - ((T' - $\mathbf{S_i}$) mod $\mathbf{F_i}$), where T' is max(T_w, $\mathbf{S_i}$). The arrival time at the final city can thus be computed by iteratively applying the *Arrival*() function to each city in succession. T_w for city i will equal *Arrival*(i) if we do not go sightseeing in city i, and *Arrival*(i) + $\mathbf{T_s}$ if we do.

Large dataset

A brute force approach will not work for the large dataset as there are now 2000 cities. However, as noted above, the time of arrival in any city is only dependent on the time that we start waiting at the previous city. This allows us to use dynamic programming to solve this problem.

One option that might come to mind is to compute m(i, j), the maximum number of cities that you can go sightseeing in if you have reached city i by time j. This can easily be expressed as a recurrence relation on smaller values of i and j. However, since the times can be as large as 10^9, this approach is not possible. Instead, we should try to compute f(i, j), the earliest time we can arrive at city i, after having gone sightseeing at exactly j different cities. For the base case of f(2, 0), this is equal to *Arrival*(2, 0) as defined above, while f(2, 1) = *Arrival*(2, $\mathbf{T_s}$).

To obtain the recurrence relation, consider a particular city i, where i > 1. You can either choose to go sightseeing there, or not. If you do, then that delays your departure time by $\mathbf{T_s}$ but adds 1 to the total number of cities in which you have gone sightseeing. This relationship can be captured as the following equation:

f(i, j) = min(*Arrival*(i, f(i - 1, j)), *Arrival*(i, f(i - 1, j - 1) + $\mathbf{T_s}$))

Once we have computed all possible f, the answer can be obtained by finding the maximum value of x such that f(**N**, x) ≤ $\mathbf{T_f}$. This solution runs in O($\mathbf{N^2}$) time.

Kick Start 2017 - Round D

Analysis: Trash

Trash Throwing: Analysis

In this problem, we will use binary search multiple times to transform an optimization problem into a decision problem. The problem asks us to find the maximal radius for a thrown circle with a center position satisfying f(x)=*a*x(x-**P**), without touching the ceiling or any obstacles.

First of all, if there is an *a* so that a circle with radius **R** could pass, then for all **R'** ≤ **R**, **R'** is also a valid solution, as we can choose the same *a*. In other words, we can binary search on **R**, and for each **R**, check whether there is a valid *a*. If there is, **R** is an acceptable answer.

How can we determine whether there is a valid parabola for a given **R**?

Let's convert the problem into a simpler form. Bob wants to throw a circle without touching any obstacles. That is, for any point on the parabola (x, f(x)), the distance from that point to the obstacle ($\mathbf{X_i}$, $\mathbf{Y_i}$) must be larger than **R**. We can reframe the problem as follows: given some obstacle circles, with the i-th circle centered at ($\mathbf{X_i}$, $\mathbf{Y_i}$) with radius **R**, Bob will throw a point such that the point does not touch/enter any circles. (We must also lower the ceiling by exactly **R** units.) This conversion does not change the final answer.

In the Small dataset, there is only one obstacle, as shown in example, and there are two ways to throw it into the trash can. One way is to throw the point over the obstacle, and the other one is to throw the point under the obstacle. Let's consider the former situation. If there is an *a* for which the parabola does not touch the obstacle circle, then for all *a'* ≤ *a*, the parabola with parameter *a'* is a valid solution. Therefore, we can binary search again (this time on *a*) and get an interval of real numbers that are valid value of *a*. The latter situation is similar. After we have considered both situations, we will have two (or maybe one, in a corner case) intervals of possible values for *a*.

After we binary search to find **R** and *a*, the problem becomes, given the parabola parameter *a* and an obstacle circle with radius **R**, determine whether the parabola will intersect the circle. We can list the equations:

$y = a(x-\mathbf{P})$

$(x - \mathbf{X_i})^2 + (y - \mathbf{Y_i})^2 = \mathbf{R^2}$

Since *a*, **P**, $\mathbf{X_i}$, $\mathbf{Y_i}$ and **R** are all known variables, we can solve the equations by Newton's method or Ferrari's algorithm. Each real root represents an intersection point of the parabola and the circle, so an *a* is acceptable if the equations do not have real roots. As for the ceiling case, since the parabola gets the maximal f(x) when x=**P**/2, if the maximal f(x) is lower than **H**-**R**, the trash will not touch the ceiling. This is enough to solve the Small dataset.

Finally, let us handle the case of multiple obstacles. We can use the same binary search mechanism. Note that in the previous solution for one obstacle, after binary search for **R**, we used another binary search to find the valid intervals for *a*. What we need to do is to find a value of *a* that is valid for all the obstacles. We can do this by masking all invalid intervals, and seeing what remains. If the invalid intervals cover all real numbers, then the value of **R** that we are considering is not acceptable.